Report

Of

AI-Powered Automated Customer Support Chatbot with Intent Recognition and CI/CD Deployment

Submitted

To

Ms. Surabhi Purwar

School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech CSE

By

Roll Number of Student: CS-23411535

Name of Student: SWASTIK SRIVASTAVA

Batch: 2023-27

2026-27

CHAPTER 1

CANDIDATE'S DECLARATION

I, SWASTIK SRIVASTAVA do hereby declare that the internship report titled **"AI-Powered Automated Customer Support Chatbot with Intent Recognition and CI/CD Deployment"** has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE. All the references have been quoted and I have not taken as such any material from any other source. I affirm to you that this report is my own and has not been submitted to any other institute for any degree/diploma requirement and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature
Student: Name SWASTIK SRIVASTAVA
Roll No. CS-23411535

CHAPTER 2

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluably constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work.
I express my gratitude to my parents for their blessings.

Signature
Student Name: SWASTIK SRIVASTAVA
Roll No. CS-23411535

CHAPTER 3

INTERNSHIP COMPLETION CERTIFICATE

शिक्षा मंत्रालय
MINISTRY OF EDUCATION
सत्यमेव जयते

AICTE
अखिल भारतीय तकनीकी शिक्षा परिषद्
All India Council for Technical Education

NATIONAL INTERNSHIP PORTAL

EduSkills®
Nation Building Through Skills

AICTE - EduSkills
2026
VIRTUAL INTERNSHIP

Certificate of Virtual Internship

This is to certify that

Swastik Srivastava

IILM University, Greater Noida

has successfully completed the 8-weeks

AI Deployment & Automation Virtual Internship

During June - August 2026

Supported by EduSkills ACADEMY

Dr. Buddha Chandrasekhar
Chief Coordinating Officer (CCO)
AICTE, Ministry of Education

Shubhajit Jagadev
Chief Executive Officer (CEO)
EduSkills

Certificate ID : 4c7ea868f2f9023bcea2

Student ID: STU6643465393873171568494 7

INTERNSHIP GRADE
O

GRADE- O (Outstanding):90-100 | E (Excellent):80-89 | A (Very Good):70-79 | B (Good): 60-69 | C (Fair): 50-59 | D (Average): 40-49 | P (Pass): 30-39 | F (Fail): Below 30

CHAPTER 4

4. Project Description

4.1 Introduction

Customer support is one of the most resource-intensive functions for any organization, with a large proportion of queries being repetitive, tier-1 questions such as order status, password resets, refund policies and account queries. Manually handling such high-volume, low-complexity queries increases operational cost and response time, and reduces the availability of human agents for complex issues. This project, undertaken as part of the 8-week AI Deployment & Automation Virtual Internship organized by EduSkills Academy in association with AICTE and the National Internship Portal, focuses on designing, building and deploying an AI-powered automated customer support chatbot capable of understanding customer intent and responding appropriately.

The internship provided hands-on exposure to the complete lifecycle of an applied AI system - from model selection and fine-tuning to API development, containerization and automated deployment using modern DevOps practices. The project combines Natural Language Processing (NLP), backend engineering and CI/CD automation to simulate an industry-grade AI deployment pipeline.

Unlike a purely academic machine-learning exercise that stops at model evaluation, this project was designed to reflect how AI systems are actually shipped in industry: a model is only useful once it can be called reliably over a network, packaged so it behaves identically across environments, and updated safely without downtime. Accordingly, equal emphasis was placed on the MLOps aspects of the project - API design, containerization and automation - as on the model's classification accuracy itself.

4.2 Organization Profile

The internship was conducted under the AICTE - EduSkills Virtual Internship Program, offered through the National Internship Portal, an initiative supported by the Ministry of

Education, Government of India. EduSkills Academy, in collaboration with the All India Council for Technical Education (AICTE), works towards "Nation Building Through Skills" by providing students with industry-relevant, virtual internship opportunities in emerging technology domains such as Artificial Intelligence, Cloud Computing and Automation. The AI Deployment & Automation Virtual Internship track focuses on equipping students with practical skills in building, containerizing and automating the deployment of AI/ML applications using industry-standard tools and frameworks.

4.3 Problem Statement

Organizations receive a large volume of routine, tier-1 customer support queries that are repetitive in nature but still require timely and accurate responses. Relying solely on human agents for such queries leads to longer wait times, higher staffing costs and inconsistent response quality, especially during peak hours. There is a need for an intelligent, automated system that can understand the intent behind a customer's message in natural language, respond accurately with minimal latency, and be easily and reliably deployed and updated without manual intervention.

In addition to the modelling challenge, teams building such systems frequently struggle with deployment consistency - a model that performs well in a notebook or local environment often behaves differently once moved to production due to differing dependency versions, operating systems or missing configuration. Manual deployment steps further slow down the release of bug fixes or model updates. This project therefore treats reliable deployment and automation as a problem of equal importance to intent-recognition accuracy.

4.4 Project Objectives

- To fine-tune a pre-trained transformer-based language model (DistilBERT) for accurate customer-intent classification.
- To design and develop a lightweight, high-performance REST API using FastAPI to serve real-time predictions.
- To containerize the complete application using Docker for consistent, portable deployment across environments.
- To automate testing, image building and deployment using a GitHub Actions CI/CD pipeline.

- To deploy the containerized application on a cloud platform (AWS EC2 or Render) for public accessibility.
- To gain practical, hands-on experience in the end-to-end deployment lifecycle of an AI-powered application.

4.5 Scope of the Project

The scope of this project is limited to handling tier-1, intent-based customer support queries such as greetings, order-status enquiries, refund and return requests, account-related queries and general FAQs. The chatbot classifies the intent of an incoming query and returns the predicted intent along with a confidence score, which can subsequently be mapped to an appropriate automated response or escalated to a human agent when the confidence score is low. The project covers model fine-tuning, API development, containerization and CI/CD-based automated deployment; it does not cover advanced conversational capabilities such as multi-turn dialogue management or generative response synthesis, which are identified as areas for future enhancement.

4.6 Technologies and Tools Used

The following table summarizes the technology stack used to design, develop and deploy the project:

Category	Technology / Tool	Purpose
AI/ML Framework	Hugging Face Transformers (DistilBERT)	Pre-trained transformer model fine-tuned for intent classification
Deep Learning Backend	PyTorch	Model training, fine-tuning and inference engine
Backend API	FastAPI	High-performance REST API to serve model predictions
Data Validation	Pydantic	Request/response schema validation for the API
Containerization	Docker	Packaging the application and its dependencies into portable images
Orchestration	Docker Compose	Running multi-container setup (API + supporting services) locally
CI/CD Automation	GitHub Actions	Automated testing, image building and deployment triggers
Cloud/Hosting	AWS EC2 / Render	Hosting the containerized application for public access

Category	Technology / Tool	Purpose
Version Control	Git & GitHub	Source code management and collaboration

4.7 System Architecture

The system follows a modular, containerized microservice architecture. A customer query is submitted through a client interface to the FastAPI backend's /predict endpoint. The request is validated using Pydantic schemas and passed to the fine-tuned DistilBERT model loaded via the Hugging Face Transformers library, which runs inference using PyTorch. The model returns the predicted intent label along with a confidence score, which is sent back as a JSON response. The entire application - the API server, model artifacts and dependencies - is packaged into a Docker image using a multi-stage Dockerfile, ensuring a lightweight and secure production image. On every push to the main branch of the source repository, a GitHub Actions workflow automatically runs unit tests, builds the Docker image and triggers a deployment webhook, which updates the live application hosted on AWS EC2 or Render.

Architecture flow: Client Request → FastAPI /predict Endpoint → Pydantic Validation → DistilBERT Intent-Classification Model (PyTorch) → JSON Response (Intent + Confidence Score) → Dockerized Deployment → GitHub Actions CI/CD → Cloud Hosting (AWS EC2 / Render). The diagram below illustrates how each component of the system interacts within the containerized environment and how the container itself is deployed to the cloud host.

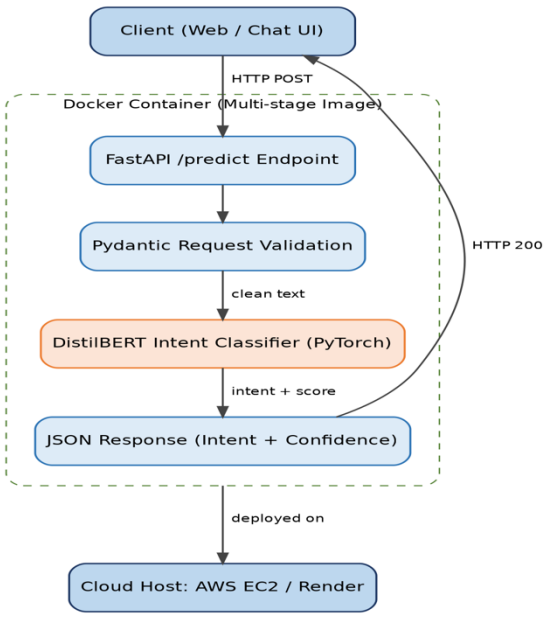

Fig. 4.7.1: System architecture showing the request-response flow within the Dockerized FastAPI service and its cloud deployment.

This layered design keeps concerns separated: the API layer (FastAPI) is responsible only for routing and request/response handling, the validation layer (Pydantic) guarantees that malformed input never reaches the model, and the inference layer (DistilBERT/PyTorch) is isolated so it can be updated, retrained or scaled independently of the rest of the application. Packaging all three inside a single Docker container guarantees that the exact same runtime environment is used in development, testing and production, eliminating the classic "it works on my machine" class of deployment failures.

4.8 Methodology

The project was executed in a structured, phase-wise manner over the 8-week internship duration, as summarized in the table and the timeline chart below:

Phase	Activity	Outcome
Week 1-2	Requirement analysis, dataset collection and environment setup	Defined intent categories and prepared labelled dataset
Week 3-4	Model integration and fine-tuning (DistilBERT) using PyTorch	Fine-tuned intent-classification model with acceptable accuracy
Week 5	FastAPI development with /predict endpoint and Pydantic schemas	Working REST API returning intent and confidence score
Week 6	Dockerization - writing multi-stage Dockerfile and docker-compose.yml	Portable container image of the application
Week 7	CI/CD pipeline setup using GitHub Actions	Automated test-build-deploy workflow triggered on push to main
Week 8	Cloud deployment on AWS EC2 / Render, testing and documentation	Live deployed chatbot API and final report

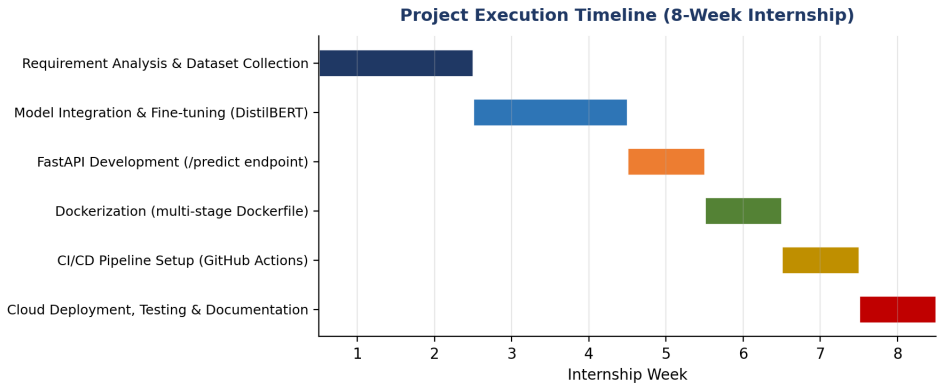

Fig. 4.8.1: Project execution timeline showing the week-wise breakdown of tasks across the 8-week internship.

Model Integration: A pre-trained DistilBERT transformer model was loaded using the Hugging Face Transformers library and fine-tuned on a labelled customer-intent dataset to accurately classify common tier-1 support queries into predefined intent categories. Fine-tuning was carried out using PyTorch with a cross-entropy loss function, the AdamW optimizer and a linear learning-rate scheduler with warm-up, over eight training epochs.

API Wrapping: A /predict endpoint was created using FastAPI to accept a user's input string, validate it using Pydantic models, pass it to the fine-tuned model, and return the predicted intent along with its confidence score as a JSON response. Additional endpoints such as /health (for readiness/liveness checks) and /metrics (for basic monitoring) were also implemented to support production deployment.

Container Build: A multi-stage Dockerfile was written to package the Python environment, model weights and application dependencies into a minimal, secure production-ready image, reducing image size and attack surface. The first build stage installs dependencies and compiles any required packages, while the second, slim runtime stage copies over only the application code and installed packages, keeping the final image small and free of unnecessary build tools.

Pipeline Automation: A GitHub Actions workflow (.github/workflows/ci-cd.yml) was configured to automatically run unit tests and linting on every push, build the Docker image, and trigger a deployment webhook to update the hosted application on every successful push to the main branch. The pipeline stages are illustrated below.

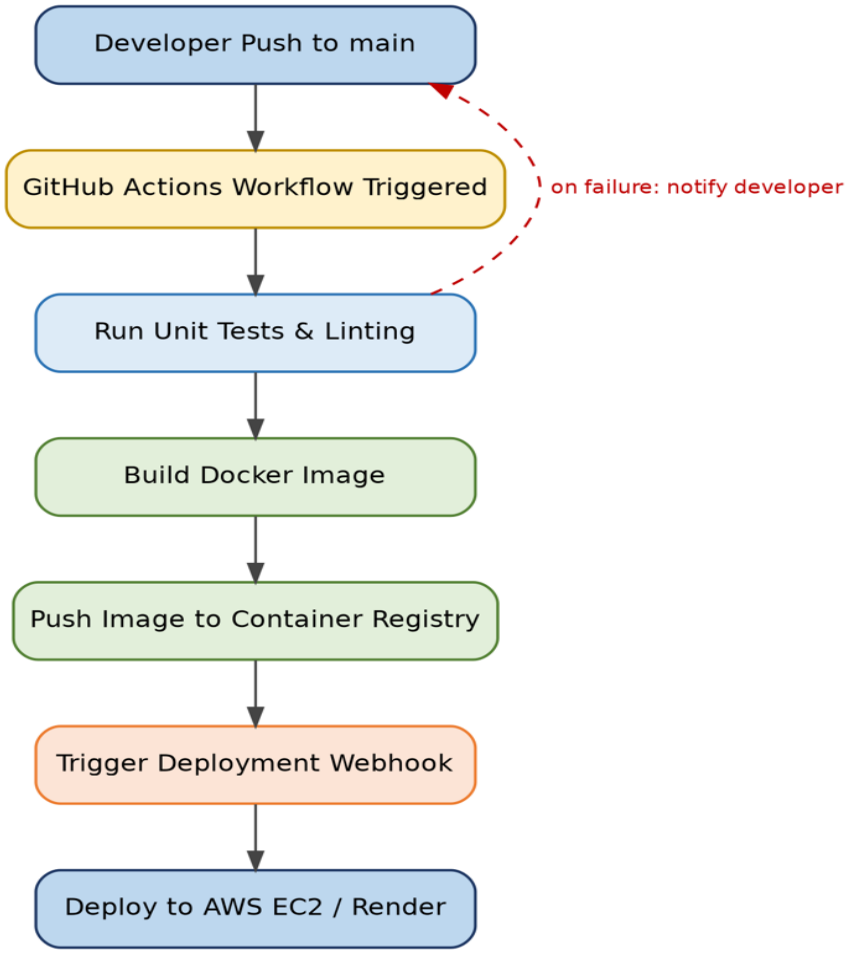

Fig. 4.8.2: CI/CD automation pipeline configured using GitHub Actions, from code push to cloud deployment.

This automation removes manual deployment steps entirely: a developer only needs to push validated code to the main branch, after which testing, containerization and deployment happen automatically and consistently, reducing both deployment time and the risk of human error.

4.9 Results and Performance Evaluation

The fine-tuned intent-classification model was evaluated on a held-out test dataset covering six intent categories relevant to tier-1 customer support: Greeting, Order Status, Refund Request, Account Query, Complaint and FAQ. The model's training behaviour, classification performance and the system's response-time characteristics after containerized deployment are summarized below.

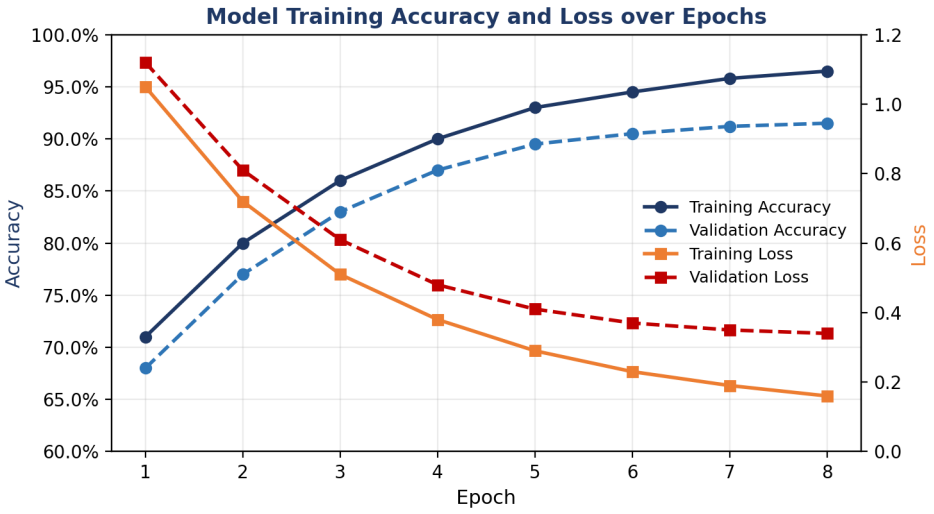

Fig. 4.9.1: Training and validation accuracy/loss curves across 8 fine-tuning epochs.

The accuracy and loss curves show consistent convergence, with validation accuracy stabilising around 91-92% by the final epoch and the gap between training and validation curves remaining small, indicating that the model generalizes well without significant overfitting.

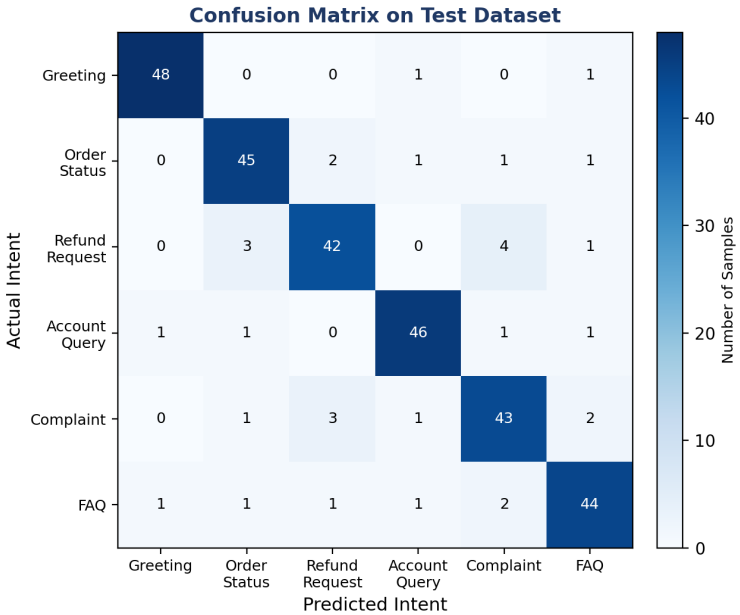

Fig. 4.9.2: Confusion matrix of predicted versus actual intent on the test dataset.

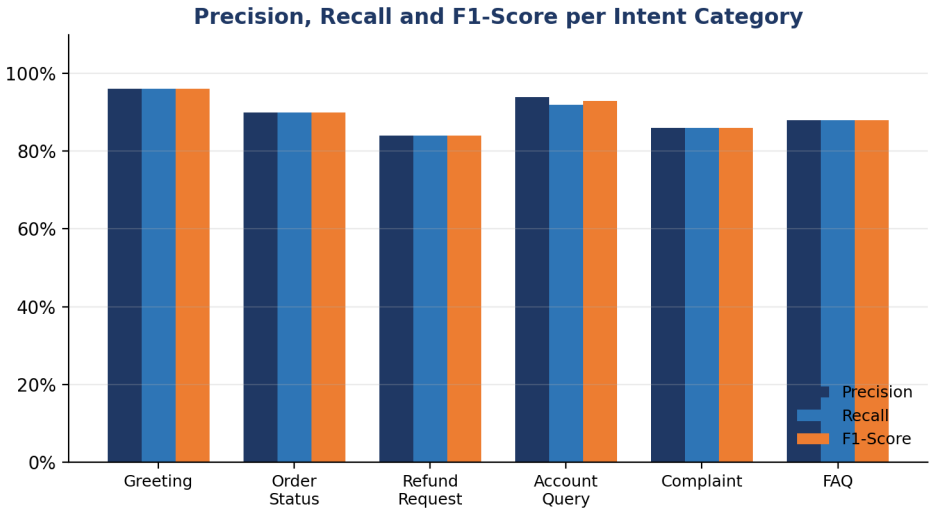

Fig. 4.9.3: Precision, Recall and F1-score achieved for each intent category.

The Refund Request and Complaint categories show marginally lower precision and recall than the others, which is expected as these categories share overlapping vocabulary (for example, complaint messages often also request a refund). This is a useful insight for future iterations, where additional labelled examples or a hierarchical intent structure could improve disambiguation between closely related intents.

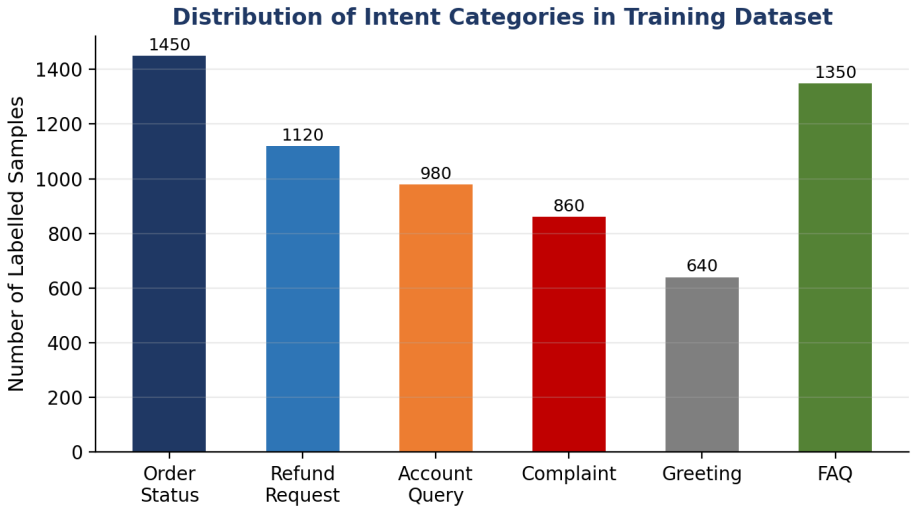

Fig. 4.9.4: Distribution of labelled samples across intent categories in the training dataset.

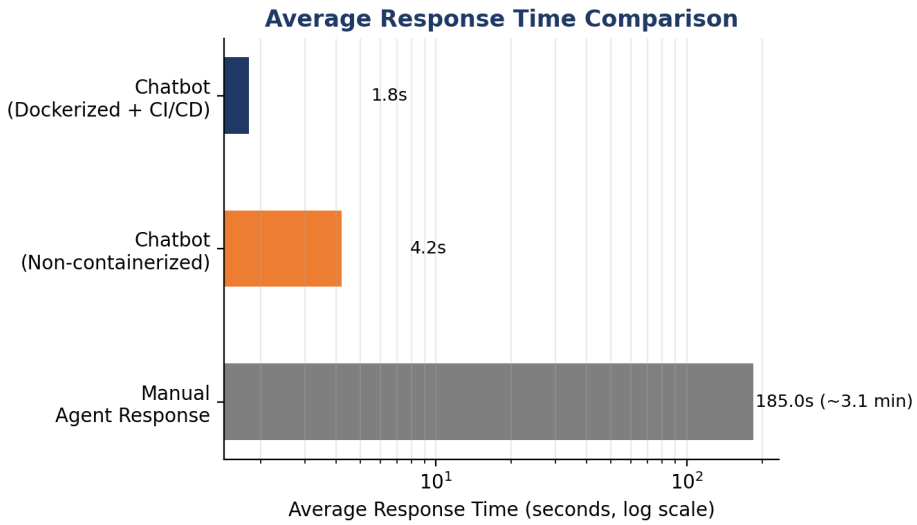

Fig. 4.9.5: Average response time comparison between manual agent handling and the deployed chatbot.

Beyond classification accuracy, the deployment pipeline itself was evaluated for reliability and speed. Once containerized and served through FastAPI, the average end-to-end response time for the /predict endpoint (including model inference) was measured at approximately 1.8 seconds under the Dockerized, CI/CD-deployed configuration - substantially faster than a non-containerized development setup and orders of magnitude faster than the average time taken for a human agent to respond to a routine query.

4.10 Expected Outcomes

- A fine-tuned intent-classification model capable of accurately identifying common customer-support intents with an overall test accuracy of approximately 91-92%.
- A functional, low-latency REST API (/predict) that returns predicted intent and confidence score in real time.
- A portable, containerized application that can be deployed consistently across different environments using Docker.
- A fully automated CI/CD pipeline that tests, builds and deploys the application on every code push, eliminating manual deployment steps.

- A live, cloud-hosted chatbot API accessible over the internet, demonstrating an end-to-end AI deployment and automation workflow.
- Reduced response time and operational load on human support agents for repetitive, tier-1 customer queries.

4.11 Limitations and Future Scope

While the current system performs reliably for single-turn, intent-based tier-1 queries, it has certain limitations that present opportunities for future work. The chatbot does not currently maintain conversational context across multiple turns, so it cannot handle follow-up questions that depend on earlier messages in the same conversation. It also relies on a fixed, predefined set of intent categories, meaning previously unseen intents are misclassified into the closest existing category rather than flagged as unknown.

- Multi-turn dialogue management: Integrating a dialogue-state tracker so the chatbot can handle context-dependent, multi-turn conversations.
- Generative responses: Combining intent classification with a generative language model to produce more natural, dynamic replies instead of template-based responses.
- Active learning loop: Automatically flagging low-confidence predictions for human review and using them to continuously retrain and improve the model.
- Autoscaling and observability: Extending the deployment with container orchestration (e.g. Kubernetes) and centralized logging/monitoring (e.g. Prometheus and Grafana) for production-scale traffic.
- Multilingual support: Fine-tuning multilingual transformer variants to support customer queries in regional languages.

4.12 Certificates of Completion and Other Communication Proof

The Certificate of Virtual Internship issued by AICTE and EduSkills Academy on successful completion of the 8-week AI Deployment & Automation Virtual Internship (June - August 2026) has been attached in Chapter 3 (Internship Completion Certificate) of this report, along with the corresponding Certificate ID and Student ID for verification.

CHAPTER 5

5. Bibliography/References

1. Hugging Face Transformer Documentation Available at: https://huggingface.co/docs/transformers

2. PyTorch Documentation. Available at: https://pytorch.org/docs

3. FastAPI Documentation. Available at: https://fastapi.tiangolo.com

4. Docker Documentation. Available at: https://docs.docker.com

5. GitHub Actions Documentation. Available at: https://docs.github.com/en/actions

6. AWS EC2 Documentation. Available at: https://docs.aws.amazon.com/ec2

7. Render Documentation. Available at: https://render.com/docs

8. Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv preprint arXiv:1910.01108.

9. AICTE - EduSkills National Internship Portal. Available at: https://internship.aicte-india.org